

Relatório do Sistema de arquivos

1. Introdução

Este relatório tem como objetivo apresentar a metodologia e os resultados obtidos durante a implementação de um sistema de arquivos baseado em inodes. Serão destacadas suas principais funcionalidades, além de uma versão alternativa que utiliza lista encadeada para armazenar as informações dos blocos de arquivos.

2. Metodologia

O sistema de arquivos foi implementado em **Python**, utilizando bibliotecas padrão para simulação e análise de desempenho. As principais ferramentas e abordagens utilizadas foram:

- **Python 3**: Linguagem escolhida pela simplicidade na prototipagem.
- **uuid**: Utilizada para geração de identificadores únicos para os inodes.
- **random**: Usada para gerar palavras aleatórias que simulam o conteúdo dos arquivos.
- **time**: Responsável pela medição do tempo de execução das operações de leitura e escrita.
- **matplotlib.pyplot**: Utilizada para gerar gráficos comparativos de desempenho.

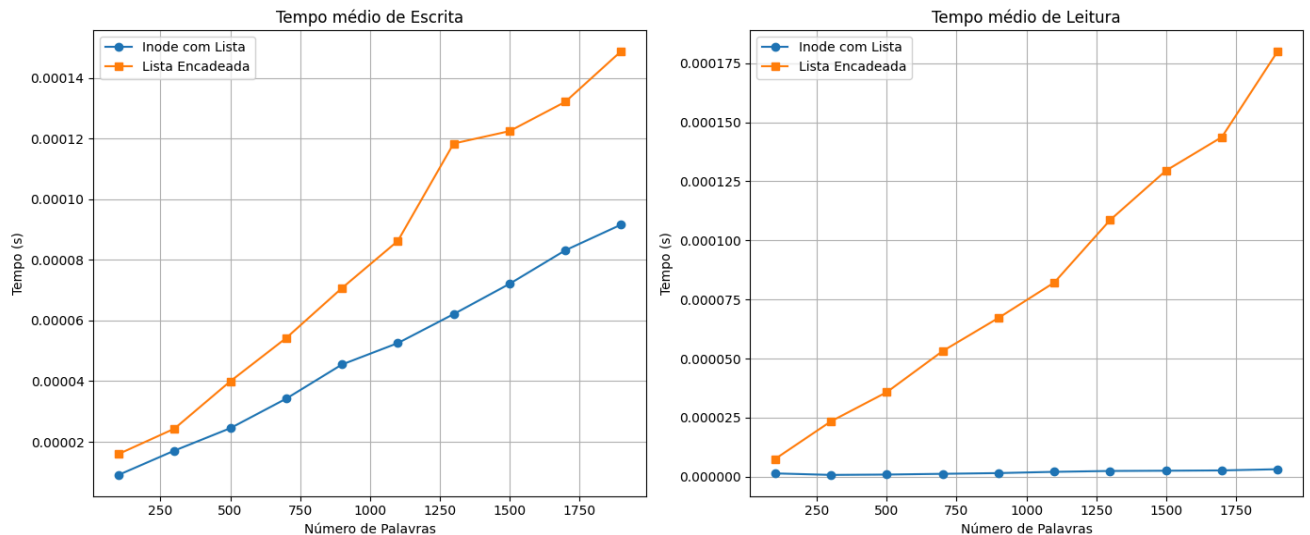
Foram implementadas duas versões do sistema: uma utilizando **lista encadeada de blocos** e outra com **array indexado via inodes**. Cada operação foi executada 10 vezes para diferentes quantidades de palavras, e a média dos tempos foi utilizada para comparar as eficiências.

3. Facilidade de Navegação e Acessos Aleatórios

Embora tanto o array indexado quanto a lista encadeada apresentem boa navegação sequencial, no quesito **acesso aleatório** a lista encadeada apresenta desvantagens significativas. Isso ocorre porque, para acessar um bloco específico, é necessário percorrer todos os blocos anteriores, o que torna a operação mais demorada e complexa. Já o array permite acesso direto e rápido a qualquer bloco.

4. Eficiência nas Operações de Leitura, Escrita e Movimentação de Arquivos

Analisando os tempos médios de escrita e leitura, observa-se que, em todas as situações testadas, o array indexado via inode apresenta melhor desempenho. Isso se evidencia especialmente em arquivos com grande quantidade de palavras ou blocos, nos quais o tempo médio de leitura e escrita é consideravelmente menor.



5. Simplicidade e Clareza da Implementação

No aspecto de clareza e simplicidade de implementação, ambos os métodos são relativamente semelhantes. No entanto, o uso de **lista encadeada** demanda um pouco mais de estrutura, exigindo, por exemplo, a criação de uma classe para representar os blocos com seus respectivos ponteiros para o próximo. Além disso, é necessário percorrer a lista inteira sempre que um novo bloco é adicionado, o que aumenta levemente a complexidade. Ainda assim, ambos os métodos apresentam uma implementação clara e compreensível.

6. Conclusão

A implementação de um sistema de arquivos baseado em inodes demonstrou ser mais eficiente em termos de desempenho, especialmente para operações de leitura e escrita em arquivos grandes. Embora a lista encadeada seja funcional e apresente uma boa navegação sequencial, ela se mostra limitada no acesso aleatório e um pouco mais complexa na implementação. Ambas as abordagens têm estrutura clara, mas o array indexado via inode se destaca pela simplicidade de acesso e melhor performance geral.

